

7. Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

Make use of the `/proc//maps` file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

OUTPUT :

```
#include <stdio.h>
#include <stdlib.h>

int global = 100;
static int s = 50;

int main()
{
    int stack = 10;
    int *heap = (int*)malloc(sizeof(int));

    printf("Code Segment : %p\n", main);
    printf("Global Segment: %p\n", &global);
    printf("Static Segment: %p\n", &s);
    printf("Heap Segment : %p\n", heap);
    printf("Stack Segment : %p\n", &stack);

    free(heap);

    return 0;
}
```

```
bhargav08@Bhargava:~/OS$ nano practical7.c
bhargav08@Bhargava:~/OS$ gcc practical7.c -o practical7
bhargav08@Bhargava:~/OS$ ./practical7
Code Segment : 0x641f795151a9
Global Segment: 0x641f79518010
Static Segment: 0x641f79518014
Heap Segment : 0x641f8454e010
Stack Segment : 0x7ffc09ac219c
```

8. Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

OUTPUT :

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *a, *b;

    a = (int*)malloc(5 * sizeof(int));
    b = (int*)calloc(5, sizeof(int));
    a = (int*)realloc(a, 10 * sizeof(int));

    free(a);
    free(b);

    return 0;
}
```

8(B):

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    int x = 100;

    if (fork() == 0)
    {
        printf("Child Before=%d\n", x);

        x = 500;

        printf("Child After=%d\n", x);
    }
    else
    {
        sleep(2);

        printf("Parent=%d\n", x);
    }

    return 0;
}
```

```
bhargav08@Bhargava:~/OS$ nano practical8A.c
bhargav08@Bhargava:~/OS$ gcc practical8A.c -o practical8A
bhargav08@Bhargava:~/OS$ ./practical8A
bhargav08@Bhargava:~/OS$ nano practical8B.c
bhargav08@Bhargava:~/OS$ gcc practical8B.c -o practical8B
bhargav08@Bhargava:~/OS$ ./practical8B
Child Before=100
Child After=500
Parent=100
```